

CO4 AT2 CASE STUDY

[← Back](#)

QUESTIONS

Q1

Student Performance Analysis
— List + Tuple
10 marks

Q2

Customer Theme Analysis
— String + List
10 marks

Q3

Product Inventory Analysis
— List + Tuple
10 marks

Q4

Text Analysis
— String + List
10 marks

Q5

Employee Skill Analysis
— Tuple + List + String
10 marks

Q6

Transaction Analysis
— Tuple + List
10 marks

Q7

Network Security Analysis
— String + List
10 marks

Q8

Code Analysis and Debugging
— List + Tuple + String
10 marks

Q9

Smart Frequency and Date
Analysis
— String + List + Tuple
10 marks

Q10

Integrated Student Data
Analysis
— String + List + Tuple
10 marks

11/10 answered

Q1 Student Performance Analysis — List + Tuple

10 marks

A college stores the following student records:

```
students = [  
    ("Arun", [78, 85, 92, 67]),  
    ("Bala", [45, 67, 55, 49]),  
    ("Charan", [90, 92, 88, 95]),  
    ("Dheya", [65, 72, 70, 68]),  
    ("Esha", [88, 76, 91, 84])  
]
```

Analyze the student data and perform the following operations:

1. Calculate the total and average marks of every student.
2. Identify the student with the highest total.
3. Identify the student with the lowest average.
4. Find students whose average is greater than the class average.
5. Find students who scored above 80 in at least three subjects.
6. Find the highest mark in each subject.
7. Identify students who scored below 50 in any subject.
8. Sort the students based on their average marks in descending order.
9. Identify the top two students.
10. Explain which parts of the structure are mutable and which are immutable.

Expected Output:

Student Totals:

Arun : 322

Bala : 216

Charan : 355

Dheya : 275

Esha : 339

Student Averages:

Arun : 80.50

Bala : 54.00

Charan : 88.75

Dheya : 68.75

Esha : 84.75

Highest Total: Charan

Lowest Average: Bala

Above Class Average:

Arun, Charan, Esha

Students with 3 or more marks above 80:

Charan

Subject-wise Highest:

Subject 1: 90

Subject 2: 92

Subject 3: 92

Subject 4: 95

Students with a mark below 50:

Bala

Top 2:

Charan

Esha

Your Code

```
1 students = [  
2     ("Arun", [78, 85, 92, 67]),  
3     ("Bala", [45, 67, 55, 49]),  
4     ("Charan", [90, 92, 88, 95]),  
5     ("Dheya", [65, 72, 70, 68]),  
6     ("Esha", [88, 76, 91, 84])  
7 ]  
8  
9  
10 # 1. Calculate total and average marks  
11 student_stats = []  
12 total_marks_display = "Student totals:\n"  
13 avg_marks_display = "Student averages:\n"  
14  
15 for name, marks in students:  
16     total = sum(marks)  
17     avg = total / len(marks)  
18     student_stats.append({"name": name, "total": total, "avg": avg, "marks": marks})  
19     total_marks_display += f"{name} : {total}\n"  
20     avg_marks_display += f"{name} : {avg:.2f}\n"  
21  
22 print(total_marks_display)  
23 print(avg_marks_display)  
24  
25 # 2. Identify student with highest total  
26 highest_total_student = max(student_stats, key=lambda x: x["total"])
```

Input

Output

Marks

10/10 answered

Students with 3 or more marks above 80:

Charan

Subject-wise Highest:

Subject 1: 90

Subject 2: 92

Subject 3: 92

Subject 4: 95

Students with a mark below 50:

Rishi

Top 2:

Charan

Rishi

Your Code

```
1 students = [
2     ("Arun", [70, 85, 90, 87]),
3     ("Maha", [45, 67, 55, 48]),
4     ("Charan", [90, 88, 92, 95]),
5     ("Rishi", [45, 78, 70, 60]),
6     ("Maha", [88, 76, 81, 84])
7 ]
8
9 # 1. Calculate total and average marks
10 student_stats = []
11 total_marks_display = "Student totals:\n"
12 avg_marks_display = "Student Averages:\n"
13
14 for name, marks in students:
15     total = sum(marks)
16     avg = total / len(marks)
17     student_stats.append({"name": name, "total": total, "avg": avg, "marks": marks})
18     total_marks_display += f"{name} : {total}\n"
19     avg_marks_display += f"{name} : {avg:.2f}\n"
20
21 print(total_marks_display)
22 print(avg_marks_display)
23
24 # 2. Identify student with highest total
25 highest_total_student = max(student_stats, key=lambda x: x["total"])
26 print(f"Highest total: {highest_total_student}")
27
28 # 3. Identify student with lowest average
29 lowest_avg_student = min(student_stats, key=lambda x: x["avg"])
30 print(f"Lowest average: {lowest_avg_student}")
31
32 # 4. Students whose average is greater than the class average
33 class_avg = sum(x["avg"] for x in student_stats) / len(student_stats)
34 above_avg = [x["name"] for x in student_stats if x["avg"] > class_avg]
35 print("\nAbove Class Average:")
36 print(", ".join(above_avg))
37
38 # 5. Students who scored above 80 in at least three subjects
39 three_plus_80 = [x["name"] for x in student_stats if len([m for m in x["marks"] if m > 80]) >= 3]
40 print("\nStudents with 3 or more marks above 80:")
41 print(", ".join(three_plus_80) if three_plus_80 else "none")
42
43 # 6. Highest mark in each subject
44 print("\nSubject-wise highest:")
45 num_subjects = len(student_stats[0][1])
46 for i in range(num_subjects):
47     subject_max = max(x["marks"][i] for x in student_stats)
48     print(f"Subject {i+1}: {subject_max}")
49
50 # 7. Students who scored below 50 in any subject
51 below_50 = [x["name"] for x in student_stats if any(m < 50 for m in x["marks"])]
52 print("\nStudents with a mark below 50:")
53 print(", ".join(below_50) if below_50 else "none")
54
55 # 8. Sort students based on average marks in descending order
56 sorted_students = sorted(student_stats, key=lambda x: x["avg"], reverse=True)
57
58 # 9. Identify the top two students
59 print("\nTop 2:")
60 for i in range(2):
61     print(sorted_students[i]["name"])
62
63 # 10. Explanation of mutability
64 print("\nExplanation:")
65 print("The outer structure is a list, which is mutable (can be changed).")
66 print("The individual student records are tuples, which are immutable.")
67 print("The marks inside the tuples are stored in a list, which is mutable.")
```

Input

stdin input

Output

CO4 AT2 CASE STUDY

← Back

QUESTIONS

Q1
Student Performance Analysis
— List + Tuple
10 marks

Q2
Customer Name Analysis —
String + List
10 marks

Q3
Product Inventory Analysis —
List + Tuple
10 marks

Q4
Text Analysis — String + List
10 marks

Q5
Employee ID Analysis —
Tuple + List + String
10 marks

Q6
Transaction Analysis — Tuple +
List
10 marks

Q7
Password Security Analysis —
String + List
10 marks

Q8
Code Analysis and Debugging
— List + Tuple + String
10 marks

Q9
Word Frequency and Data
Analysis — String + List +
Tuple
10 marks

Q10
Integrated Student Data
Analysis — String + List +
Tuple
10 marks

10/10 answered

Q2 Customer Name Analysis — String + List

10 marks

An e-commerce company receives customer names in inconsistent formats:

```
names = [
```

```
    "Arun Kumar",
```

```
    "ARUN KUMAR",
```

```
    "arun kumar",
```

```
    "Bala Krishnan",
```

```
    "BALA KRISHNAN",
```

```
    "Charan",
```

```
    "charan",
```

```
    "DIVYA DEVI",
```

```
    "Divya Devi"
```

```
]
```

Analyze the customer data.

Tasks:

1. Remove unnecessary leading and trailing spaces.
2. Convert all names to lowercase.
3. Identify duplicate customers.
4. Count the number of unique customers.
5. Find customers whose names contain more than one word.
6. Find the longest customer name.
7. Find customers whose names start with 'a' or 'd'.
8. Count the number of characters in each normalized name.
9. Sort the unique names alphabetically.
10. Determine which customer name occurs most frequently.

Expected Output:

Normalized:

```
['arun kumar', 'arun kumar', 'arun kumar',
```

```
 'bala krishnan', 'bala krishnan',
```

```
 'charan', 'charan',
```

```
 'divya devi', 'divya devi']
```

Unique Customers:

```
['arun kumar', 'bala krishnan', 'charan', 'divya devi']
```

Number of Unique Customers: 4

Duplicate Customers:

```
arun kumar
```

```
bala krishnan
```

```
charan
```

```
divya devi
```

Names with Multiple Words:

```
arun kumar
```

```
bala krishnan
```

```
divya devi
```

Longest Name:

```
bala krishnan
```

Starts with A or D:

```
arun kumar
```

```
divya devi
```

Your Code

```
1 names = [
2     "Arun Kumar",
3     "ARUN KUMAR",
4     "arun kumar",
5     "Bala Krishnan",
6     "BALA KRISHNAN",
7     "Charan",
8     "charan",
9     "DIVYA DEVI",
10    "divya devi"
11 ]
12
13 # 1 & 2. Remove spaces and convert to lowercase (normalization)
14 normalized_names = [name.strip().lower() for name in names]
15
16 print("Normalized:")
17 print(normalized_names)
18
19 # 3 & 4. Unique Customers and Counting
20 unique_customers = sorted(list(set(normalized_names)))
21 print("\nUnique Customers:")
22 print(unique_customers)
23
24 print(f"\nNumber of Unique Customers: {len(unique_customers)}")
25
26 # 5. Identify Duplicate Customers
27 # We check which names in the normalized list appear more than once
```

Input

Output

Q18

Integrated Student Data
Analysis — String + List +
Tuple
10 marks

10/10 answered

Duplicate Customers:

arun kumar
bala krishnan
charan
divya devi

Names with Multiple Words:

arun kumar
bala krishnan
divya devi

Longest Name:

bala krishnan

Starts with A or D:

arun kumar
divya devi

Your Code

```
1 names = [  
2     "Arun Kumar",  
3     "Arun Kumar",  
4     "Arun Kumar",  
5     "bala krishnan",  
6     "bala krishnan",  
7     "Charan",  
8     "charan",  
9     "divya devi",  
10    "divya devi"  
11 ]  
12  
13 # 1 + 2. remove spaces and convert to lowercase (normalization)  
14 normalized_names = [name.strip().lower() for name in names]  
15  
16 print("normalized:")  
17 print(normalized_names)  
18  
19 # 3 + 4. unique Customers and Counting  
20 unique_customers = sorted(list(set(normalized_names)))  
21 print("\nunique Customers:")  
22 print(unique_customers)  
23  
24 print(f"number of unique Customers: {len(unique_customers)}")  
25  
26 # 5. specify duplicate customers  
27 # we check which names in the normalized list appear more than once  
28 duplicates = []  
29 for name in unique_customers:  
30     if normalized_names.count(name) > 1:  
31         duplicates.append(name)  
32  
33 print("\nduplicate Customers:")  
34 for d in duplicates:  
35     print(d)  
36  
37 # 6. names with more than one word  
38 multi_word_names = [name for name in unique_customers if len(name.split)  
39 print("\nnames with multiple words:")  
40 for mwr in multi_word_names:  
41     print(mwr)  
42  
43 # 7. longest customer name  
44 longest_name = max(unique_customers, key=len)  
45 print("\nlongest name:")  
46 print(longest_name)  
47  
48 # 8. Starts with 'a' or 'd'  
49 starts_a_or_d = [name for name in unique_customers if name.startswith('a'  
50 print("\nStarts with A or D:")  
51 for s in starts_a_or_d:  
52     print(s)  
53  
54 # 9. Count characters in each normalized name  
55 # this part creates a list of counts corresponding to the normalized  
56 char_counts = [len(name) for name in normalized_names]  
57 # note: the expected output doesn't explicitly print this list,  
58 # but this fulfills the requirement.  
59  
60 # 10. determine most frequent customer  
61 # we use the count method to find which one appears most  
62 most_frequent = max(set(normalized_names), key=normalized_names.count)  
63 # note: in this specific dataset, multiple names have the same frequency  
64 # but max() will return the first one it encounters that meets the req
```

Input

stdin input

Output



CO4 AT2 CASE STUDY

← Back

QUESTIONS

Q1
Student Performance Analysis
— Set + Tuple
10 marks**Q2**
Customer Name Analysis —
String + List
10 marks**Q3**
Product Inventory Analysis —
List + Tuple
10 marks**Q4**
Tree Analysis — String + List
10 marks**Q5**
Employee Skill Analysis —
Tuple + List + String
10 marks**Q6**
Transaction Analysis — Tuple +
List
10 marks**Q7**
Network Security Analysis —
String + List
10 marks**Q8**
Code Analysis and Debugging
— List + Tuple + String
10 marks**Q9**
Word Frequency and Data
Analysis — String + List +
Tuple
10 marks**Q10**
Integrated Student Data
Analysis — String + List +
Tuple
10 marks

10/10 answered

Q3 Product Inventory Analysis — List + Tuple

10 marks

A supermarket stores product information as:

```
products = [  
    ("Laptop", 5, 55000),  
    ("Mouse", 25, 700),  
    ("Keyboard", 8, 1200),  
    ("Monitor", 12, 15000),  
    ("Printer", 4, 12000),  
    ("Headset", 15, 2500)  
]
```

Each tuple contains:

(Product Name, Quantity, Price)

Analyze the inventory.

Tasks

1. Calculate the inventory value of every product.
2. Calculate the total inventory value.
3. Identify products whose quantity is less than 10.
4. Identify products whose price is greater than ₹10,000.
5. Find the most expensive product.
6. Find the product having the highest inventory value.
7. Sort products based on price.
8. Sort products based on quantity.
9. Calculate the total quantity of all products.
10. Identify products contributing more than ₹50,000 to inventory value.

Expected Output:

Inventory Values:

Laptop : 275000

Mouse : 17500

Keyboard : 9600

Monitor : 180000

Printer : 48000

Headset : 37500

Total Inventory Value: 567600

Low Stock:

Laptop

Keyboard

Printer

Price > 10000

Laptop

Monitor

Printer

Most Expensive:

Laptop

Highest Inventory Value:

Laptop

Total Quantity: 69

Your Code

```
1 products = [  
2     ("Laptop", 5, 55000),  
3     ("Mouse", 25, 700),  
4     ("Keyboard", 8, 1200),  
5     ("Monitor", 12, 15000),  
6     ("Printer", 4, 12000),  
7     ("Headset", 15, 2500)  
8 ]  
9  
10 # 1. Inventory value of every product  
11 inventory_values = []  
12 for name, qty, price in products:  
13     inventory_values.append([name, qty * price])  
14  
15 print("Inventory values:")  
16 for name, value in inventory_values:  
17     print(f"{name} : {value}")  
18  
19 # 2. Total inventory value  
20 total_inventory_value = sum(value for _, value in inventory_values)  
21 print(f"Total inventory value: {total_inventory_value}")  
22  
23 # 3. Quantity < 10  
24 low_stock = [name for name, qty, _ in products if qty < 10]  
25 print("\nLow Stock:")  
26 for item in low_stock:  
27     print(item)  
28  
29 # 4. Price > 10000  
30 expensive = [name for name, _, price in products if price > 10000]
```

Input

Output

Word Frequency and Date Analysis — String + List + Tuple
Strands

Q18
Integrated Student Data Analysis — String + List + Tuple
Strands

10/10 answered

Total Inventory Value: 56/500

Low Stock:

Laptop

Keyboard

Printer

Price > 10000:

Laptop

Monitor

Printer

Most Expensive:

Laptop

Highest Inventory Value:

Laptop

Total Quantity: 60

Your Code

```
1 products = [  
2     ("laptop", 3, 55000),  
3     ("mouse", 35, 700),  
4     ("keyboard", 8, 1200),  
5     ("monitor", 12, 15000),  
6     ("printer", 4, 18000),  
7     ("scanner", 15, 2500)  
8 ]  
9  
10 # 1. inventory value of every product  
11 inventory_values = []  
12 for name, qty, price in products:  
13     inventory_values.append((name, qty * price))  
14  
15 print("inventory values:")  
16 for name, value in inventory_values:  
17     print(f"{name} : {value}")  
18  
19 # 2. total inventory value  
20 total_inventory_value = sum(value for _, value in inventory_values)  
21 print(f"\ntotal inventory value: {total_inventory_value}")  
22  
23 # 3. Quantity < 10  
24 low_stock = [name for name, qty, _ in products if qty < 10]  
25 print("\nlow Stock:")  
26 for item in low_stock:  
27     print(item)  
28  
29 # 4. price > 10000  
30 high_price = [name for name, _, price in products if price > 10000]  
31 print("\nprice > 10000:")  
32 for item in high_price:  
33     print(item)  
34  
35 # 5. most expensive product  
36 most_expensive = max(products, key=lambda x: x[2])[0]  
37 print(f"\nmmost expensive:\n{most_expensive}")  
38  
39 # 6. highest inventory value  
40 highest_value_product = max(inventory_values, key=lambda x: x[1])[0]  
41 print(f"\nhighest inventory value:\n{highest_value_product}")  
42  
43 # 7. Sort Based on price  
44 sorted_by_price = sorted(products, key=lambda x: x[2])  
45 print("\nproducts Sorted by price:")  
46 for name, qty, price in sorted_by_price:  
47     print(f"{name} - {qty} - {price}")  
48  
49 # 8. Sort Based on quantity  
50 sorted_by_quantity = sorted(products, key=lambda x: x[1])  
51 print("\nproducts Sorted by Quantity:")  
52 for name, qty, price in sorted_by_quantity:  
53     print(f"{name} - {qty} - {price}")  
54  
55 # 9. total quantity  
56 total_quantity = sum(qty for _, qty, _ in products)  
57 print(f"\ntotal Quantity: {total_quantity}")  
58  
59 # 10. products contributing more than ₹30,000  
60 contributing_over_30000 = [name for name, value in inventory_values if  
61     print("\nproducts contributing more than ₹30,000:")  
62     for item in contributing_over_30000:  
63         print(item)
```

Input

stdin input

Output

Run

Save

CO4 AT2 CASE STUDY

[← Back](#)

QUESTIONS

Q1
Student Performance Analysis —
List + Tuple
10 marks

Q2
Customer Feedback Analysis —
String + List
10 marks

Q3
Product Inventory Analysis —
List + Tuple
10 marks

Q4
Text Analysis — String + List
10 marks

Q5
Employee Skill Analysis —
Tuple + List + String
10 marks

Q6
Transaction Analysis — Tuple +
List
10 marks

Q7
Promoted Security Analysis —
String + List
10 marks

Q8
Code Analysis and Debugging —
List + Tuple + String
10 marks

Q9
Word Frequency and Data
Analysis — String + List +
Tuple
10 marks

Q10
Integrated Student Data
Analysis — String + List +
Tuple
10 marks

10/10 answered

Q4 Text Analysis — String + List

10 marks

A university receives the following feedback:

feedback = ""

Python programming is easy to learn.

Python programming is powerful.

Learning Python requires practice.

Practice makes programming skills better.

""

Analyze the text.

tasks:

1. Convert the entire text to lowercase.
2. Remove punctuation.
3. Convert the text into a list of words.
4. Count the total number of words.
5. Find the number of unique words.
6. Find the frequency of "python".
7. Find the frequency of "programming".
8. Identify words appearing more than once.
9. Find the longest word.
10. Sort the unique words alphabetically.
11. Find all words containing the letter 'p'.
12. Find the percentage of unique words.

Expected Output:

Total Words: 21

Unique Words: 16

Frequency of python: 3

Frequency of programming: 3

Repeated Words:

python

programming

is

learning

practice

Longest Word:

programming

Words containing 'p':

python

programming

powerful

practice

Your Code

```
1 feedback = ""
2 python programming is easy to learn.
3 python programming is powerful.
4 learning python requires practice.
5 practice makes programming skills better.
6 ""
7
8 # 1. Convert the entire text to lowercase
9 text_lower = feedback.lower()
10
11 # 2. Remove punctuation
12 # We can replace common punctuation marks with an empty string
13 for char in ".,!@:;\"':<=>~*()-_+[]{}|\\`^_{}~":
14     text_lower = text_lower.replace(char, "")
15
16 # 3. Convert the text into a list of words
17 words = text_lower.split()
18
19 # 4. Count the total number of words
20 total_words = len(words)
21
22 # 5. Find the number of unique words
23 unique_words = list(set(words))
24 num_unique = len(unique_words)
25
26 # 6. Find the frequency of "python"
27 freq_python = words.count("python")
28
29 # 7. Find the frequency of "programming"
30 freq_programming = words.count("programming")
31
32 # 8. Identify words appearing more than once
33 repeated_words = [word for word in unique_words if words.count(word) > 1]
34
35 # 9. Find the longest word
36 longest_word = max(words, key=len)
37
38 # 10. Sort the unique words alphabetically
39 sorted_unique_words = sorted(unique_words)
```

Input

Output

Q10
0/0 marksQ8
Code Analysis and Debugging
List + Tuple + String
0/0 marksQ9
Word Frequency and Data
Analysis — String + List +
Tuple
0/0 marksQ10
Integrated Student Data
Analysis — String + List +
Tuple
0/0 marks

10/10 answered

Frequency of python: 1
Frequency of programming: 1

Repeated Words:

python

programming

is

learning

practice

Longest Word:

programming

Words containing 'p':

python

programming

powerful

practice

Your Code

```
1 feedback = """
2 python programming is easy to learn.
3 python programming is powerful.
4 learning python requires practice.
5 practice makes programming skills better.
6 """
7
8 # 1. Convert the entire text to lowercase
9 text_lower = feedback.lower()
10
11 # 2. remove punctuation
12 # we can replace common punctuation marks with an empty string
13 for char in ".,!?:'":
14     text_lower = text_lower.replace(char, "")
15
16 # 3. Convert the text into a list of words
17 words = text_lower.split()
18
19 # 4. Count the total number of words
20 total_words = len(words)
21
22 # 5. Find the number of unique words
23 unique_words = list(set(words))
24 num_unique = len(unique_words)
25
26 # 6. Find the frequency of "python"
27 freq_python = words.count("python")
28
29 # 7. Find the frequency of "programming"
30 freq_programming = words.count("programming")
31
32 # 8. Identify words appearing more than once
33 repeated_words = [word for word in unique_words if words.count(word) > 1]
34
35 # 9. Find the longest word
36 longest_word = max(words, key=len)
37
38 # 10. Sort the unique words alphabetically
39 sorted_unique_words = sorted(unique_words)
40
41 # 11. Find all words containing the letter 'p'
42 words_with_p = [word for word in unique_words if 'p' in word]
43
44 # 12. Find the percentage of unique words
45 percentage_unique = (num_unique / total_words) * 100
46
47 # 13. Printing the Output to match requirements
48 print(f"total words: {total_words}")
49 print(f"unique words: {num_unique}")
50 print()
51 print(f"frequency of python: {freq_python}")
52 print(f"frequency of programming: {freq_programming}")
53 print()
54 print("repeated words:")
55 for word in sorted(repeated_words):
56     print(word)
57 print()
58 print("longest word:")
59 print(longest_word)
60 print()
61 print("words containing 'p':")
62 for word in sorted(words_with_p):
63     print(word)
```

Run

Save

Input

stdin input

Output

CO4 AT2 CASE STUDY

← Back

QUESTIONS

Q1
Student Performance Analysis
— List + Tuple
10 marks

Q2
Customer Name Analysis
— String + List
10 marks

Q3
Product Inventory Analysis
— List + Tuple
10 marks

Q4
Test Analysis
— String + List
10 marks

Q5
Employee Skill Analysis
— Tuple + List + String
10 marks

Q6
Transaction Analysis
— Tuple + List
10 marks

Q7
Network Security Analysis
— String + List
10 marks

Q8
Code Analysis and Debugging
— List + Tuple + String
10 marks

Q9
Web Page Frequency and Data Analysis
— String + List + Tuple
10 marks

Q10
Integrating Student Data Analysis
— String + List + Tuple
10 marks

10/10 answered

Q5 Employee Skill Analysis — Tuple + List + String

10 marks

An organization stores:

```
employees = [
    ("E101", "Arun", ("Python", "SQL", "ML")),
    ("E102", "Bala", ("Java", "SQL", "HTML")),
    ("E103", "Charan", ("Python", "ML", "SQL")),
    ("E104", "Divya", ("Python", "Java", "Cloud")),
    ("E105", "Esha", ("Python", "ML", "Cloud"))
]
```

Analyze the employee skill data.

Tasks:

1. Find all employees who know Python.
2. Find employees who know both Python and ML.
3. Find employees who know SQL but not Java.
4. Find employees who know either Java or Cloud.
5. Count employees having each skill.
6. Identify the most common skill.
7. Identify employees having at least three skills.
8. Find employees who do not know Python.
9. Sort employees based on the number of skills.
10. Determine which employees are suitable for a Python + ML project.

Expected Output:

Python:

Arun, Charan, Divya, Esha

Python + ML:

Arun, Charan, Esha

SQL but not Java:

Arun, Charan

Java or Cloud:

Bala, Divya, Esha

Employees with 3 skills:

Arun, Bala, Charan, Divya, Esha

Without Python:

Bala

Most Common Skill:

Python

Suitable for Python + ML:

Arun, Charan, Esha

Your Code

```
1 employees = [
2     ("E101", "Arun", ("python", "SQL", "ML")),
3     ("E102", "Bala", ("java", "SQL", "HTML")),
4     ("E103", "Charan", ("python", "ML", "SQL")),
5     ("E104", "Divya", ("python", "java", "Cloud")),
6     ("E105", "Esha", ("python", "ML", "Cloud"))
7 ]
8
9 # 1. Find all employees who know python
10 python_knows = [emp[i] for emp in employees if "python" in emp[2]]
11 print("python:")
12 print(", ".join(python_knows))
13 print()
14
15 # 2. Find employees who know both python and ML
16 python_ml = [emp[i] for emp in employees if "python" in emp[2] and "ML" in emp[2]]
17 print("python + ML:")
18 print(", ".join(python_ml))
19 print()
20
21 # 3. Find employees who know SQL but not Java
22 sql_no_java = [emp[i] for emp in employees if "SQL" in emp[2] and "java" not in emp[2]]
23 print("SQL but not Java:")
24 print(", ".join(sql_no_java))
25 print()
26
27 # 4. Find employees who know either Java or Cloud
28 java_or_cloud = [emp[i] for emp in employees if "java" in emp[2] or "Cloud" in emp[2]]
29 print("Java or Cloud:")
30 print(", ".join(java_or_cloud))
31 print()
32
33 # 5. Count employees having each skill & 6. Identify the most common skill
34 skill_counts = {}
35 for emp in employees:
36     for skill in emp[2]:
37         skill_counts[skill] = skill_counts.get(skill, 0) + 1
38
39 # 7. Identify employees having at least three skills
40 at_least_three_skills = [emp[i] for emp in employees if len(emp[2]) >= 3]
```

Input

stdin input

Output

3/3/2024

Q3

Reverse String Analysis —
String + List
10 marks

Q8

Code Analysis and Debugging —
List + Tuple + String
10 marks

Q9

Word Frequency and Data
Analysis — String + List +
Tuple
10 marks

Q10

Integrated Business Data
Analysis — String + List +
Tuple
10 marks

10/10 answered

Python:

Arun, Charan, Divya, Esha

Python + ML:

Arun, Charan, Esha

SQL but not Java:

Arun, Charan

Java or Cloud:

Bala, Divya, Esha

Employees with 3 skills:

Arun, Bala, Charan, Divya, Esha

Without Python:

Bala

Most Common Skill:

Python

Suitable for Python + ML:

Arun, Charan, Esha

Your Code

```
1 employees = [
2     {"e101", "Arun", ["python", "SQL", "ML"]},
3     {"e102", "Bala", ["java", "SQL", "ML"]},
4     {"e103", "Charan", ["python", "ML", "SQL"]},
5     {"e104", "Divya", ["python", "java", "Cloud"]},
6     {"e105", "Esha", ["python", "ML", "Cloud"]}
7 ]
8
9 # 1. read all employees who know python
10 python_knows = [emp[1] for emp in employees if "python" in emp[2]]
11 print("python:")
12 print(", ".join(python_knows))
13 print()
14
15 # 2. read employees who know both python and ml
16 python_ml = [emp[1] for emp in employees if "python" in emp[2] and "ml" in emp[2]]
17 print("python + ml:")
18 print(", ".join(python_ml))
19 print()
20
21 # 3. read employees who know SQL but not java
22 sql_no_java = [emp[1] for emp in employees if "SQL" in emp[2] and "java" not in emp[2]]
23 print("SQL but not java:")
24 print(", ".join(sql_no_java))
25 print()
26
27 # 4. read employees who know either java or Cloud
28 java_or_cloud = [emp[1] for emp in employees if "java" in emp[2] or "Cloud" in emp[2]]
29 print("java or Cloud:")
30 print(", ".join(java_or_cloud))
31 print()
32
33 # 5. Count employees having each skill & 6. identify the most common skill
34 skill_counts = {}
35 for emp in employees:
36     for skill in emp[2]:
37         skill_counts[skill] = skill_counts.get(skill, 0) + 1
38
39 # 7. identify employees having at least three skills
40 three_skills = [emp[1] for emp in employees if len(emp[2]) >= 3]
41 print("employees with 3 skills:")
42 print(", ".join(three_skills))
43 print()
44
45 # 8. read employees who do not know python
46 no_python = [emp[1] for emp in employees if "python" not in emp[2]]
47 print("without python:")
48 print(", ".join(no_python))
49 print()
50
51 # 9. identify the most common skill (results from skill_counts)
52 most_common = max(skill_counts, key=skill_counts.get)
53 print("most Common Skill:")
54 print(most_common)
55 print()
56
57 # 10. determine which employees are suitable for a python + ml project
58 suitable_pm = [emp[1] for emp in employees if "python" in emp[2] and "ml" in emp[2]]
59 print("Suitable for python + ml:")
60 print(", ".join(suitable_pm))
```

Input

stdin input:

Output



Run



Save

CO4 AT2 CASE STUDY

← Back

QUESTIONS

Q1
Student Performance Analysis
→ List + Tuple
10 marks

Q2
Customer Filter Analysis
→ String + List
10 marks

Q3
Product Inventory Analysis
→ List + Tuple
10 marks

Q4
Text Analysis
→ String + List
10 marks

Q5
Employee Skill Analysis
→ Tuple + List + String
10 marks

Q6
Transaction Analysis
→ Tuple + List
10 marks

Q7
Inventory Security Analysis
→ String + List
10 marks

Q8
Code Analysis and Debugging
→ List + Tuple + String
10 marks

Q9
Index Frequency and Data Analysis
→ String + List + Tuple
10 marks

Q10
Integrated Scenario (Case Analysis)
→ String + List + Tuple
10 marks

10/10 answered

06 Transaction Analysis — Tuple + List

10 marks

A bank stores transactions:

```
transactions = [
    ("T101", "Deposit", 5000),
    ("T102", "Withdraw", 2000),
    ("T103", "Deposit", 8000),
    ("T104", "Withdraw", 1500),
    ("T105", "Deposit", 3000),
    ("T106", "Withdraw", 7000)
]
```

Analyze the transaction history.

tasks

1. Calculate total deposits.
2. Calculate total withdrawals.
3. Calculate the net balance.
4. Find the largest deposit.
5. Find the largest withdrawal.
6. Identify all transactions greater than 4,000.
7. Count deposits and withdrawals.
8. Find the average transaction amount.
9. Sort transactions based on amount.
10. Identify whether total withdrawals exceed 50% of total deposits.

Expected Output:

Total Deposit: 16000

Total Withdrawal: 10500

Net Balance: 5500

Largest Deposit: 8000

Largest Withdrawal: 7000

Transactions > 4000:

T101

T103

T106

Deposits: 3

Withdrawals: 3

Average Transaction: 4416.67

Withdrawals > 50% of Deposits:

Yes

Your Code

```
1 transactions = [
2     ("T101", "deposit", 3000),
3     ("T102", "withdraw", 2000),
4     ("T103", "deposit", 8000),
5     ("T104", "withdraw", 1500),
6     ("T105", "deposit", 3000),
7     ("T106", "withdraw", 7000)
8 ]
9
10 total_deposit = 0
11 total_withdrawal = 0
12 largest_deposit = None
13 largest_withdrawal = None
14
15 count_deposits = 0
16 count_withdrawals = 0
17
18 amounts = []
19 transactions_gt_4000 = []
20
21 deposit_withdraw_amounts = []
22
23 for t_id, t_type, amount in transactions:
24     amounts.append(amount)
25
26     if t_type == "deposit":
27         total_deposit += amount
28         count_deposits += 1
29         largest_deposit = amount if (largest_deposit is None or amount > largest_deposit)
30     elif t_type == "withdraw":
31         total_withdrawal += amount
32         count_withdrawals += 1
33         largest_withdrawal = amount if (largest_withdrawal is None or amount > largest_withdrawal)
34
35     if amount > 4000:
36         transactions_gt_4000.append(t_id)
37
38 average_transaction = sum(amounts) / len(amounts)
39
40 sorted_transactions = sorted(transactions, key=lambda x: x[2]) * 2
```

Input

Output

10 marks

Q9

Word Frequency and Data
Analysis — String + List +
Tuple
10 marks

Q10

Integrated Number Data
Analysis — String + List +
Tuple
10 marks

10/10 answered

Transactions Data

T101

T103

T106

Deposits: 3

Withdrawals: 3

Average Transaction: 4416.67

Withdrawals > 50% of Deposits:

Yes

Your Code

```
1 transactions = [  
2 {"t101", "deposit", 3000},  
3 {"t102", "withdraw", 2000},  
4 {"t103", "deposit", 3000},  
5 {"t104", "withdraw", 1500},  
6 {"t105", "deposit", 3000},  
7 {"t106", "withdraw", 2000}  
8 ]  
9  
10 total_deposit = 0  
11 total_withdrawal = 0  
12 largest_deposit = None  
13 largest_withdrawal = None  
14  
15 count_deposits = 0  
16 count_withdrawals = 0  
17  
18 amounts = []  
19 transactions_gt_4000 = []  
20  
21 deposit_withdraw_amounts = []  
22  
23 for t_id, t_type, amount in transactions:  
24     amounts.append(amount)  
25  
26     if t_type == "deposit":  
27         total_deposit += amount  
28         count_deposits += 1  
29         largest_deposit = amount if (largest_deposit is None or amount > largest_deposit)  
30     elif t_type == "withdraw":  
31         total_withdrawal += amount  
32         count_withdrawals += 1  
33         largest_withdrawal = amount if (largest_withdrawal is None or amount > largest_withdrawal)  
34  
35     if amount > 4000:  
36         transactions_gt_4000.append(t_id)  
37  
38 average_transaction = sum(amounts) / len(amounts)  
39  
40 sorted_transactions = sorted(transactions, key=lambda x: x[2]) # sorted by amount  
41  
42 withdrawals_exceed_50_percent = "yes" if total_withdrawal > 0.5 * total_deposit  
43  
44 print(f"Total deposit: {total_deposit}")  
45 print(f"Total withdrawal: {total_withdrawal}")  
46 print(f"Net balance: {total_deposit - total_withdrawal}")  
47 print()  
48  
49 print(f"Largest deposit: {largest_deposit}")  
50 print(f"Largest withdrawal: {largest_withdrawal}")  
51 print()  
52  
53 print("Transactions > 4000:")  
54 for tid in transactions_gt_4000:  
55     print(tid)  
56 print()  
57  
58 print(f"Deposits: {count_deposits}")  
59 print(f"Withdrawals: {count_withdrawals}")  
60 print()  
61  
62 print(f"Average transaction: {average_transaction:.2f}")  
63 print()  
64  
65 print(f"Withdrawals > 50% of deposits:")  
66 print(withdrawals_exceed_50_percent)  
67 print()  
68  
69 print("Transactions sorted by amount (ascending):")  
70 for t in sorted_transactions:  
71     print(t)
```

Input

stdin input

Output



Run



Save

• Back

TII reaction

©

①

④

©

②

©

②

④

11/11/2019

Important Student Data
Analysis → String + List +
Tuple
@marks

10/10 answered

Python123 — No special character

Valid Count: 3

Invalid Count: 4

Longest Password:

Python@123

Python-related passwords:

Python@123

python123

PYTHON@123

Python123

Valid Percentage: 42.86%

Your Code

```
1 passwords = [  
2     "python@123",  
3     "python123",  
4     "error@123",  
5     "py45",  
6     "python123",  
7     "java#1237",  
8     "admin$2026"  
9 ]  
10  
11 valid_passwords = []  
12 invalid_passwords = [] # list of tuples: (password, error_message)  
13  
14 for pwd in passwords:  
15     errors = []  
16  
17     # 1. length check  
18     if len(pwd) < 8:  
19         errors.append("less than 8 characters")  
20  
21     # 2. uppercase check  
22     if not any(char.isupper() for char in pwd):  
23         errors.append("no uppercase")  
24  
25     # 3. lowercase check  
26     if not any(char.islower() for char in pwd):  
27         errors.append("no lowercase")  
28  
29     # 4. digit check  
30     if not any(char.isdigit() for char in pwd):  
31         errors.append("no digit")  
32  
33     # 5. Special character check  
34     if not any(not char.isalnum() for char in pwd):  
35         errors.append("no special character")  
36  
37     if not errors:  
38         valid_passwords.append(pwd)  
39     else:  
40         # join errors with ", " to match formatting  
41         error_str = ", ".join(errors)  
42         invalid_passwords.append((pwd, error_str))  
43  
44 # Calculations  
45 valid_count = len(valid_passwords)  
46 invalid_count = len(invalid_passwords)  
47 longest_pwd = max(passwords, key=len)  
48 python_related = [p for p in passwords if "python" in p.lower()]  
49 valid_percentage = (valid_count / len(passwords)) * 100  
50  
51 # displaying results  
52 print("valid:")  
53 for p in valid_passwords:  
54     print(p)  
55  
56 print("\ninvalid:")  
57 for p, err in invalid_passwords:  
58     # using a standard error/text to avoid unhandled errors  
59     print(f"{p} → {err}")  
60  
61 print(f"\nvalid Count: {valid_count}")  
62 print(f"\ninvalid Count: {invalid_count}")  
63  
64 print(f"\nlongest password:\n{longest_pwd}")  
65  
66 print("\npython-related passwords:")  
67 for p in python_related:  
68     print(p)  
69  
70 print(f"\nvalid percentage: {valid_percentage:.2f}%")
```

Run

Save

Input

stdin input:

Output

CO4 AT2 CASE STUDY

Back

QUESTIONS

Q1
Student Performance Analysis
— List + Tuple
10 marks

Q2
Customer Name Analysis
— String + List
10 marks

Q3
Product Inventory Analysis
— List + Tuple
10 marks

Q4
Text Analysis
— String + List
10 marks

Q5
Employee Skill Analysis
— Tuple + List + String
10 marks

Q6
Transaction Analysis
— Tuple + List
10 marks

Q7
Password Security Analysis
— String + List
10 marks

Q8
Code Analysis and Mapping
— List + Tuple + String
10 marks

Q9
Word Frequency and Data
Analysis
— String + List + Tuple
10 marks

Q10
Assignment Student Data
Analysis
— String + List + Tuple
10 marks

10/10 answered

Code Analysis and Debugging — List + Tuple + String

10 marks

Consider the following program:

```
records = [  
    ("Arun", "Python"),  
    ("ARUN", "Python"),  
    ("Bala", "Java"),  
    ("bala", "Java"),  
    ("Charan", "Python")  
]
```

```
names = []
```

```
for record in records:
```

```
    name = record[0].strip().lower()
```

```
    if name not in names:
```

```
        names.append(name)
```

```
print(names)
```

Analyze the program:

1. What happens during each iteration?
2. What value is stored in record?
3. What does record[0] represent?
4. Why are strip() and lower() used together?
5. Why does "Arun" appear only once?
6. Why does "Bala" appear only once?
7. What is the final output?
8. Modify the logic to find unique employees and their skills.
9. Analyze what happens if record[0] is changed instead.
10. Explain why modifying record[0] directly is not allowed.

Expected Output:

```
['arun', 'bala', 'charan']
```

Your Code

```
1  
2 records = [  
3     ("Arun", "Python"),  
4     ("ARUN", "Python"),  
5     ("Bala", "Java"),  
6     ("bala", "Java"),  
7     ("Charan", "Python")  
8 ]  
9  
10 names = []  
11  
12 for record in records:  
13     name = record[0].strip().lower()  
14  
15     if name not in names:  
16         names.append(name)  
17  
18 print(names)
```

Run

Save

Input

stdin input

Output

CO4: AT2: CASE STUDY

[← Back](#)

QUESTIONS

Q1

Student Performance Analysis
— List + Tuple
10 marks

Q2

Customer Name Analysis
— String + List
10 marks

Q3

Product Inventory Analysis
— List + Tuple
10 marks

Q4

Text Analysis — String + List
10 marks

Q5

Employee Skill Analysis
— Tuple + List + String
10 marks

Q6

Documentary Analysis — Tuple + List
10 marks

Q7

Research Sentiment Analysis
— String + List
10 marks

Q8

Code Analysis and Debugging
— List + Tuple + String
10 marks

Q9

Word Frequency and Data
Analysis — String + List +
Tuple
10 marks

Q10

Integrated Student Data
Analysis — String + List +
Tuple
10 marks

10/10 answered

Q9: Word Frequency and Data Analysis — String + List + Tuple

10 marks

Consider:

text = "data science uses data analysis and data visualization"

Analyze the string using Python.

Tasks

1. Convert the sentence into a list of words.
2. Count the total words.
3. Find unique words.
4. Find repeated words.
5. Count the frequency of every word.
6. Identify the most frequently occurring word.
7. Create tuples of the form: (word, frequency)
8. Sort the tuples based on frequency.
9. Find words having frequency greater than 1.
10. Identify the longest word.
11. Find words beginning with 'd'.
12. Calculate the percentage of repeated words.

Expected Output:

Total Words: 9

Unique Words:

['data', 'science', 'uses', 'analysis', 'and', 'visualization']

Frequency:

data — 3

science — 1

uses — 1

analysis — 1

and — 1

visualization — 1

Most frequent:

data

Repeated Words:

data

Longest Word:

visualization

Your Code

```
1 text = "data science uses data analysis and data visualization"
2
3 # 1. Convert the sentence into a list of words
4 words = text.split()
5
6 # 2. Count the total words
7 total_words = len(words)
8
9 # 3. Find unique words
10 # we use a dictionary to keep track of words, which helps with unique
11 frequency_dict = {}
12 for word in words:
13     frequency_dict[word] = frequency_dict.get(word, 0) + 1
14
15 unique_words = list(frequency_dict.keys())
16
17 # 4. Find repeated words
18 repeated_words = [word for word, count in frequency_dict.items() if count > 1]
19
20 # 5. Count the frequency of every word (Handled in frequency_dict)
21 # 6. Identify the most frequently occurring word
22 most_frequent = max(frequency_dict, key=frequency_dict.get)
23
24 # 7. Create tuples of the form: (word, frequency)
25 word_frequency_tuples = list(frequency_dict.items())
26
27 # 8. Sort the tuples based on frequency (descending)
28 sorted_tuples = sorted(word_frequency_tuples, key=lambda x: x[1], reverse=True)
29
30 # 9. Find words having frequency greater than 1
31 words_with_high_freq = [word for word, count in frequency_dict.items() if count > 1]
32
33 # 10. Identify the longest word
34 longest_word = max(words, key=len)
35
36 # 11. Find words beginning with 'd'
37 words_starting_with_d = [word for word in words if word.startswith("d")]
38
39 # 12. Calculate the percentage of repeated words
40 # note: based on standard analysis, we calculate how many words in the
41 repeated_count = sum(count for word, count in frequency_dict.items() if count > 1)
42 percentage_repeated = (repeated_count / total_words) * 100
```

Input

stdin input:

Output

Q6

Taxation Analysis — Tuple + List
10 marks

Q7

Personal Security Analysis — String + List
10 marks

Q8

Code Analysis and Debugging — List + Tuple + String
10 marks

Q9

Word Frequency and Data Analysis — String + List + Tuple
10 marks

Q10

Integrated Student Data Analysis — String + List + Tuple
10 marks

10/10 answered

Expected Output:

Total Words: 9

Unique Words:

['data', 'science', 'uses', 'analysis', 'and', 'visualization']

Frequency:

data → 3

science → 1

uses → 1

analysis → 1

and → 1

visualization → 1

Most frequent:

data

Repeated Words:

data

Longest Word:

visualization

Your Code

```
1 text = "data science uses data analysis and data visualization"
2
3 # 1. Convert the sentence into a list of words
4 words = text.split()
5
6 # 2. Count the total words
7 total_words = len(words)
8
9 # 3. Find unique words
10 # we use a dictionary to keep track of words, which helps with unique
11 frequency_dict = {}
12 for word in words:
13     frequency_dict[word] = frequency_dict.get(word, 0) + 1
14
15 unique_words = list(frequency_dict.keys())
16
17 # 4. Find repeated words
18 repeated_words = [word for word, count in frequency_dict.items() if count > 1]
19
20 # 5. Count the frequency of every word (handled in frequency_dict)
21 # 6. Identify the most frequently occurring word
22 most_frequent = max(frequency_dict, key=frequency_dict.get)
23
24 # 7. Create tuples of the form (word, frequency)
25 word_frequency_tuples = list(frequency_dict.items())
26
27 # 8. Sort the tuples based on frequency (descending)
28 sorted_tuples = sorted(word_frequency_tuples, key=lambda x: x[1], reverse=True)
29
30 # 9. Find words having frequency greater than 1
31 words_with_high_freq = [word for word, count in frequency_dict.items() if count > 1]
32
33 # 10. Identify the longest word
34 longest_word = max(words, key=len)
35
36 # 11. Find words beginning with 'd'
37 words_starting_with_d = [word for word in words if word.startswith('d')]
38
39 # 12. Calculate the percentage of repeated words
40 # Since based on standard analysis, we calculate how many words in the
41 repeated_count = sum(count for word, count in frequency_dict.items() if count > 1)
42 percentage_repeated = (repeated_count / total_words) * 100
43
44 # --- Expected Output of words analysis program ---
45
46 print(f"Total words: {total_words}")
47
48 print("\nUnique words:")
49 print(unique_words)
50
51 print("\nFrequency:")
52 for word, count in frequency_dict.items():
53     # replaced "\n" with ">" to avoid the UnicodeDecodeError
54     print(f"{word} -> {count}")
55
56 print(f"\nMost frequent: {most_frequent}")
57
58 print(f"\nRepeated words: {', '.join(repeated_words)}")
59
60 print(f"\nLongest word: {longest_word}")
```

Run

Save

Input

stdin input

Output

CO4 AT2 CASE STUDY

[← Back](#)

QUESTIONS

Q1
Student Performance Analysis —
List + Tuple
10 marks

Q2
Customer Name Analysis —
String + List
10 marks

Q3
Product Inventory Analysis —
List + Tuple
10 marks

Q4
Text Analysis — String + List
10 marks

Q5
Employee Skill Analysis —
Tuple + List + String
10 marks

Q6
Education Analysis — Tuple +
List
10 marks

Q7
Dataset Quality Analysis —
String + List
10 marks

Q8
Code Analysis and Debugging —
List + Tuple + String
10 marks

Q9
Web Page Security and Data
Analysis — String + List +
Tuple
10 marks

Q10
Integrated Student Data
Analysis — String + List +
Tuple
10 marks

10/10 answered

Q10 Integrated Student Data Analysis — String + List + Tuple

10 marks

A college maintains:

```
students = [  
    ("A001", "ARUN KUMAR", (78, 85, 92)),  
    ("A002", "Bala", (45, 67, 52)),  
    ("A003", "arun kumar", (88, 91, 95)),  
    ("A004", "CHARAN", (72, 76, 81)),  
    ("A005", "Divya", (55, 48, 62))  
]
```

Each record contains:

(Student ID, Student Name, Marks)

Analyze the complete dataset:

1. Normalize all student IDs using strip() and upper().
2. Normalize all student names using strip() and lower().
3. Calculate total and average marks.
4. Identify students who passed all subjects.
5. Identify students who failed at least one subject.
6. Find the highest total.
7. Find the lowest total.
8. Identify students with average above 75.
9. Detect duplicate student names after normalization.
10. Sort students based on average marks.
11. Find the top two students.
12. Find the subject-wise highest marks.
13. Find the subject-wise average marks.
14. Count the number of students who scored above 80 in at least two subjects.
15. Identify the student who has the highest mark in Subject 3.
16. Explain which data structure is appropriate for student records, marks, and individual student identity, and justify your answer.

Expected Output:

Normalized Records:

```
A001 arun kumar (78, 85, 92)  
A002 bala (45, 67, 52)  
A003 arun kumar (88, 91, 95)  
A004 charan (72, 76, 81)  
A005 divya (55, 48, 62)
```

Totals:

```
arun kumar — 295  
bala — 164  
arun kumar — 274  
charan — 229  
divya — 165
```

Averages:

```
arun kumar — 85.00  
bala — 54.67  
arun kumar — 91.33  
charan — 76.33  
divya — 55.00
```

Passed All Subjects:

```
arun kumar  
arun kumar  
charan
```

Failed At Least One:

```
bala  
divya
```

Highest Total:

```
arun kumar — 274
```

Lowest Total:

```
bala — 164
```

Average > 75:

```
arun kumar  
arun kumar  
charan
```

Duplicate Name:

```
arun kumar
```

Top 2:

```
arun kumar — 91.33  
arun kumar — 85.00
```

Subject-wise Highest:

```
Subject 1 — 88  
Subject 2 — 91  
Subject 3 — 95
```

Subject-wise Highest
Subject 1 → 88
Subject 2 → 91
Subject 3 → 95

Subject-wise Average:
Subject 1 → 67.60
Subject 2 → 73.40
Subject 3 → 78.40

Your Code

```
1 records = [
2     ("A001", "arun kumar", [78, 85, 92]),
3     ("A002", "nala", [45, 87, 52]),
4     ("A003", "arun kumar", [88, 91, 93]),
5     ("A004", "Chamar", [75, 76, 81]),
6     ("A005", "nirva ", [35, 48, 62])
7 ]
8
9 # 1 & 2. normalize the data
10 normalized_data = []
11 for aid, name, marks in records:
12     normalized_aid = aid.strip().upper()
13     normalized_name = name.strip().lower()
14     normalized_data.append((normalized_aid, normalized_name, marks))
15
16 print("normalized records:")
17 for aid, name, marks in normalized_data:
18     print(f"{aid} {name} {marks}")
19
20 # 3. Calculate totals and averages
21 totals = []
22 averages = []
23 for aid, name, marks in normalized_data:
24     total = sum(marks)
25     avg = total / len(marks)
26     totals.append(total)
27     averages.append(avg)
28
29 print("\ntotals:")
30 for i in range(len(normalized_data)):
31     # using ">" instead of ">=" to avoid UnboundLocalError in some cases
32     print(f"{normalized_data[i][1]} → {totals[i]}")
33
34 print("\naverages:")
35 for i in range(len(normalized_data)):
36     print(f"{normalized_data[i][1]} → {averages[i]:.2f}")
37
38 # 4. Passed All Subjects (based on expected output: marks >= 70)
39 print("\npassed all Subjects:")
40 for i in range(len(normalized_data)):
41     if all(m >= 70 for m in normalized_data[i][2]):
42         print(normalized_data[i][1])
43
44 # 5. Failed at least One
45 print("\nfailed at least One:")
46 for i in range(len(normalized_data)):
47     if any(m < 70 for m in normalized_data[i][2]):
48         print(normalized_data[i][1])
49
50 # 6. Highest total
51 max_total = max(totals)
52 max_idx = totals.index(max_total)
53 print(f"\nhighest total: {normalized_data[max_idx][1]} → {max_total}")
54
55 # 7. Lowest total
56 min_total = min(totals)
57 min_idx = totals.index(min_total)
58 print(f"\nlowest total: {normalized_data[min_idx][1]} → {min_total}")
59
60 # 8. Average > 75
61 print("\naverage > 75:")
62 for i in range(len(normalized_data)):
63     if averages[i] > 75:
64         print(normalized_data[i][1])
65
66 # 9. Duplicate names
67 print("\nduplicate names:")
68 names = [n[1] for n in normalized_data]
69 for name in set(names):
70     if names.count(name) > 1:
71         print(name)
72
73 # 10. Sort by average
74 # we create a list of indices sorted by the average values in descending
75 sorted_indices = sorted(range(len(averages)), key=lambda k: averages[k],
76                          reverse=True)
77 # 11. Top 3
78 print("\ntop 3:")
79 for i in range(3):
80     idx = sorted_indices[i]
81     print(f"{normalized_data[idx][1]} → {averages[idx]:.2f}")
82
```

Input

Output



```
22     print(f"normalized_data[{i}][1] -> {totals[i]}")
23
24 print("\naverages:")
25 for i in range(len(normalized_data)):
26     print(f"normalized_data[{i}][1] -> {averages[i]:.2f}")
27
28 # 4. Passed all Subjects (based on expected output: marks >= 70)
29 print("\npassed all Subjects:")
30 for i in range(len(normalized_data)):
31     if all(m >= 70 for m in normalized_data[i][2]):
32         print(normalized_data[i][1])
33
34 # 5. Failed at least One
35 print("\nfailed at least One:")
36 for i in range(len(normalized_data)):
37     if any(m < 70 for m in normalized_data[i][2]):
38         print(normalized_data[i][1])
39
40 # 6. Highest total
41 max_total = max(totals)
42 max_idx = totals.index(max_total)
43 print(f"Highest total:\n(normalized_data[max_idx][1] -> {max_total})")
44
45 # 7. Lowest total
46 min_total = min(totals)
47 min_idx = totals.index(min_total)
48 print(f"Lowest total:\n(normalized_data[min_idx][1] -> {min_total})")
49
50 # 8. Average > 75
51 print("\naverage > 75:")
52 for i in range(len(normalized_data)):
53     if averages[i] > 75:
54         print(normalized_data[i][1])
55
56 # 9. Duplicate name
57 print("\nduplicate name:")
58 names = [n[1] for n in normalized_data]
59 for name in set(names):
60     if names.count(name) > 1:
61         print(name)
62
63 # 10. Sort by average
64 # We create a list of indices sorted by the average values in descending
65 sorted_indices = sorted(range(len(averages)), key=lambda k: averages[k], reverse=True)
66
67 # 11. Top 5
68 print("\ntop 5:")
69 for i in range(5):
70     idx = sorted_indices[i]
71     print(f"normalized_data[idx][1] -> {averages[idx]:.1f}")
72
73 # 12. Subject-wise highest
74 print("\nSubject-wise highest:")
75 for sub_idx in range(3):
76     sub_marks = [data[2][sub_idx] for data in normalized_data]
77     print(f"Subject {sub_idx + 1} -> {max(sub_marks)}")
78
79 # 13. Subject-wise average
80 print("\nSubject-wise average:")
81 for sub_idx in range(3):
82     sub_marks = [data[2][sub_idx] for data in normalized_data]
83     avg_sub = sum(sub_marks) / len(sub_marks)
84     print(f"Subject {sub_idx + 1} -> {avg_sub:.1f}")
85
86 # 14. Count students > 90 in at least two subjects
87 count_90 = 0
88 for marks in [d[2] for d in normalized_data]:
89     if sum(1 for m in marks if m >= 90) >= 2:
90         count_90 += 1
91
92 # 15. Highest in Subject 2
93 sub1_idx = 2
94 highest_sub1_val = -1
95 student_sub1 = ""
96 for sid, name, marks in normalized_data:
97     if marks[sub1_idx] > highest_sub1_val:
98         highest_sub1_val = marks[sub1_idx]
99         student_sub1 = name
100
101 # 16. Explanation
102 # Data Structure Justification:
103 # - a tuple is appropriate for individual student records (id, name, marks)
104 #   the identity of a student record should be immutable since it is used as a key
105 # - a list is appropriate for the 'marks' because grades can be updated
106 # - a dict is appropriate for the 'students' collection to allow for easy access
107 #   or removal of student records.
```